路径控制

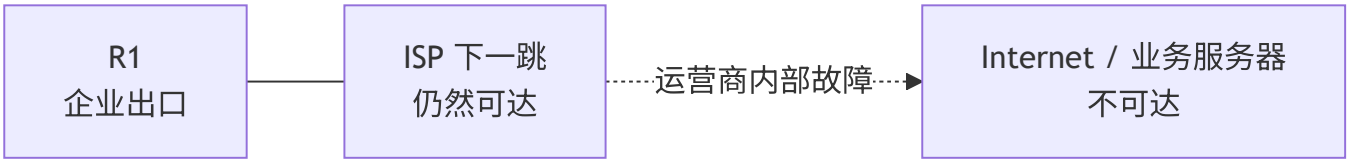
1. IP SLA + Track

IP SLA 是 Cisco 设备上的主动探测功能。可以先把它理解成：路由器自己定时去 ping、访问或测试一个目标，然后把测试结果告诉别的功能使用。

注意：IP SLA 本身不改路由。它只负责“测”。真正让路由生效或失效的，通常是 Track。

1.1 为什么需要 IP SLA

普通静态路由和动态路由，很多时候只能感知“直连接口或邻居是否正常”。但真实网络里，经常出现这种故障：



此时 R1 到 ISP 下一跳可能还能 ARP，接口也没有 Down，静态默认路由仍然存在，但互联网或业务目标已经访问不了。

IP SLA 解决的是这个问题：不要只看本地链路是否 Up，而是主动检测远端目标是否可达。

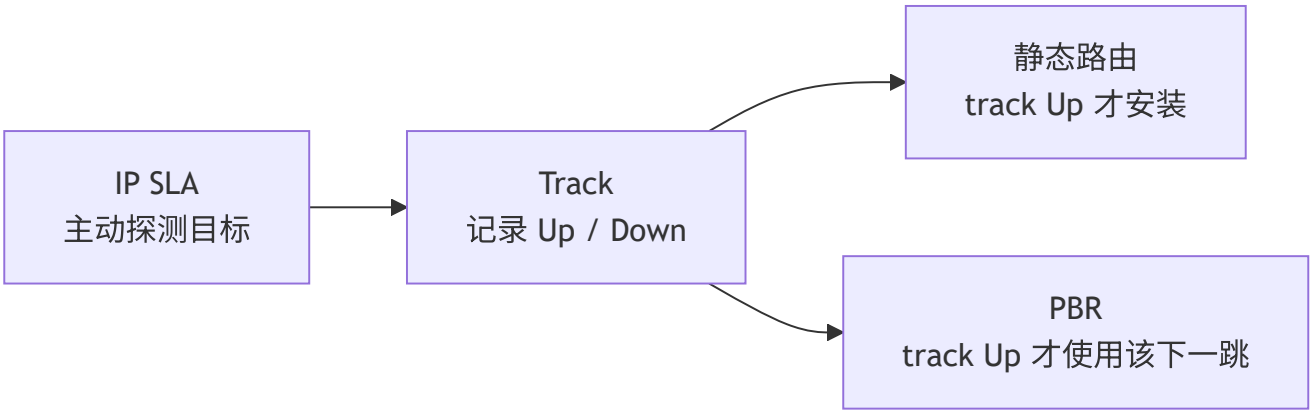
常见用途：

用途	说明
双出口主备	主出口探测失败后，撤销主默认路由
PBR 下一跳检测	策略路由下一跳不可达时，切到备用下一跳
业务可达性检测	探测 DNS、服务器、网关或业务地址
延迟/抖动检测	配合更复杂的 SLA 策略观察质量

1.2 IP SLA、Track、路由之间的关系

这三个东西要分开理解。

组件	作用	例子
IP SLA	负责主动探测	每 5 秒 ping 一次 `8.8.8.8`
Track	把探测结果变成 Up/Down 状态	探测成功就是 Up，失败就是 Down
被控制对象	根据 Track 状态决定是否生效	静态路由、PBR 下一跳等



一句话：

```
IP SLA 负责探测，Track 负责判断状态，路由/PBR 负责改变路径。
```

1.3 做 IP SLA 前先确定三件事

IP SLA 配置不能一上来就写命令。先把三件事想清楚：

问题	要确定什么	配错后的现象
探测什么	探测下一跳、DNS、业务服务器， 还是运营商给的探测地址	探测目标不代表真实业务， 导致误切或不切
从哪里探测	探测报文从主出口、备出口，还是某个 Loopback 发出	源地址不对，回包回不来，Track 一直 Down
控制什么	控制静态默认路由、PBR 下一跳， 还是其他对象	IP SLA Up/Down 了， 但路径没有变化

最容易混乱的是“探测流量本身走哪条路”。例如双 ISP 场景下，如果用 IP SLA 探测 `8.8.8.8`，主默认路由撤销后，探测报文可能被备份默认路由带到 ISP2。这样会出现一种奇怪现象：

```
ISP1 实际坏了 -> 主默认路由撤销 -> 探测报文改走 ISP2 -> 探测又成功 -> 主默认路由恢复 -> 再次失败 -> 反
```

所以探测公网目标时，通常要让探测报文固定从主出口出去。常见做法：

```
ip sla 10
 icmp-echo 8.8.8.8 source-interface Ethernet0/0

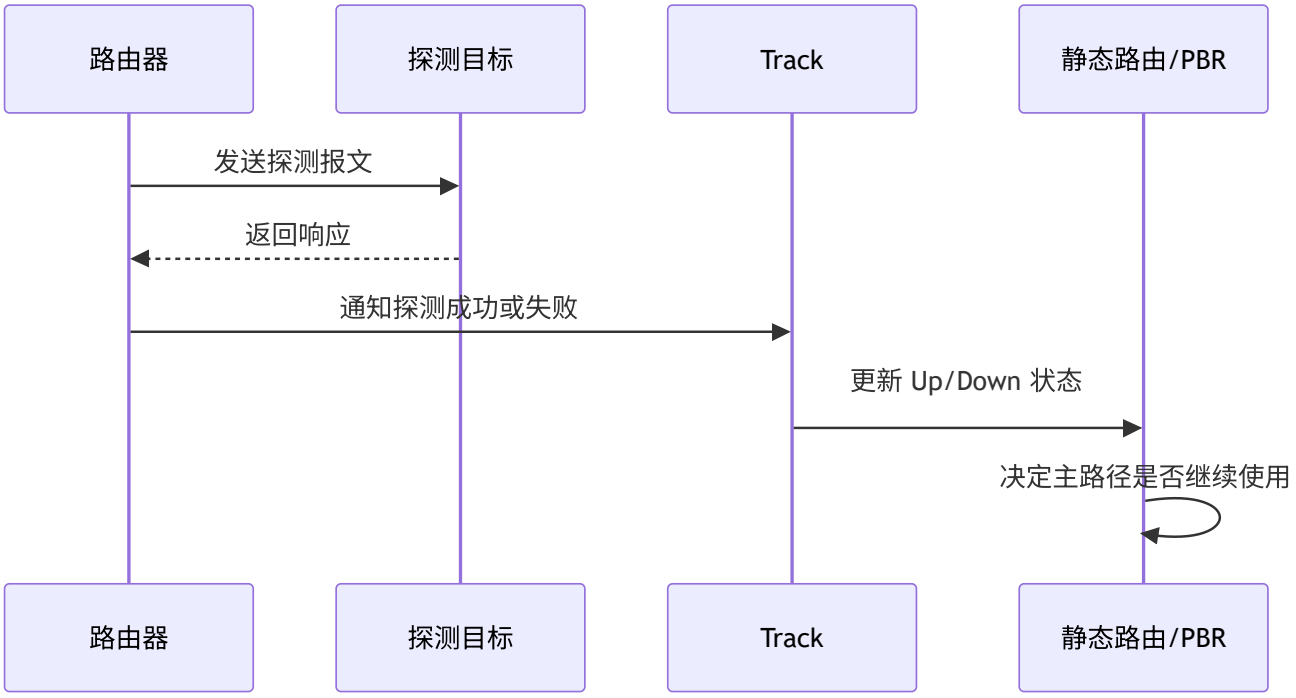
ip route 8.8.8.8 255.255.255.255 203.0.113.1
```

含义：

- `source-interface Ethernet0/0` 指定探测报文使用主出口接口地址作为源地址。
- 到探测目标 `8.8.8.8/32` 的主机路由固定指向 ISP1 下一跳。
- 默认路由切到 ISP2 后，IP SLA 探测仍然尝试从 ISP1 方向探测，用来判断 ISP1 是否恢复。

注意：如果探测目标是用户真实会访问的地址，加主机路由会让这个目标一直偏向主出口。生产环境最好选择专门的探测地址，或者使用运营商提供的 SLA 探测地址。

1.4 IP SLA 基本工作流程



最常见的探测方式是 ICMP Echo，也就是让路由器自己 ping 某个目标。

```
ip sla 10
 icmp-echo 8.8.8.8 source-interface Ethernet0/0
 frequency 5
 timeout 1000
ip sla schedule 10 life forever start-time now
```

逐行解释：

命令	含义
`ip sla 10`	创建编号为 10 的探测任务
`icmp-echo 8.8.8.8`	使用 ICMP Echo 探测 `8.8.8.8`
`source-interface Ethernet0/0`	指定探测报文从哪个接口发出
`frequency 5`	每 5 秒探测一次
`timeout 1000`	1 秒内没有响应就认为这次探测失败
`ip sla schedule`	启动探测任务
`life forever`	一直运行
`start-time now`	立即开始

`frequency` 和 `timeout` 要配合看。一般 `timeout` 要小于 `frequency`，否则上一次探测还没超时，下一次探测又开始，排障时会很乱。

```
frequency 5
timeout 1000
```

含义是每 5 秒探测一次，每次最多等待 1 秒。不是 1 秒切换，也不是 5 秒一定切换；真正切换还要看 Track 是否配置了 `delay down/up`。

创建 Track：

```
track 10 ip sla 10 reachability
```

含义：Track 10 关注 IP SLA 10 的可达性结果。

把 Track 绑定到静态路由：

```
ip route 0.0.0.0 0.0.0.0 203.0.113.1 track 10
```

含义：只有 Track 10 为 Up 时，这条默认路由才进入路由表。Track 10 Down 时，这条路由会被撤销。

最终状态链可以这样看：

```
IP SLA Return Code: OK      -> Track Up   -> 主路由安装
IP SLA Return Code: Timeout -> Track Down -> 主路由撤销
```

查看时也按这个顺序：

```
show ip sla statistics 10
show track 10
show ip route 0.0.0.0
```

IP SLA 常见探测类型：

类型	示例	适合检测什么
ICMP Echo	`icmp-echo 8.8.8.8`	基础 IP 可达性，课堂最常用
TCP Connect	`tcp-connect 10.1.1.10 443`	目标服务器端口是否可连接
UDP Jitter	`udp-jitter 10.1.1.10 16384`	语音、实时业务的延迟、抖动、丢包
DNS	解析指定域名	DNS 服务是否可用
HTTP	访问指定 URL	简单 Web 可用性检测

路径控制里最常用的是 ICMP Echo 和 TCP Connect。ICMP Echo 只能证明目标能回 ICMP；如果业务是 HTTPS，TCP Connect 到 443 更贴近业务。

不同 IOS 对 DNS、HTTP、Jitter 等探测类型的支持和命令细节可能不同。课堂路径切换实验建议先掌握 ICMP Echo 和 TCP Connect，其他类型作为扩展认识。

TCP Connect 示例：

```
ip sla 20
 tcp-connect 10.1.1.10 443 source-interface Ethernet0/0
 frequency 5
 timeout 1000
ip sla schedule 20 life forever start-time now

track 20 ip sla 20 reachability
```

这个探测不只是看 `10.1.1.10` 能不能 ping 通，而是看它的 TCP 443 端口能不能建立连接，更适合模拟 HTTPS 业务可达性。

IP SLA 常用配置命令补充

IP SLA 配置可以分成四步看：

1. 创建探测任务
2. 定义探测方式和目标
3. 设置频率、超时、阈值等参数
4. `schedule` 启动任务，再用 `track` 调用结果

常用命令：

命令	作用	说明
`ip sla <id>`	创建 IP SLA 任务	`<id>` 只在本设备本地有意义
`icmp-echo <target>`	ICMP Echo 探测	最常见，类似设备自己 ping 目标
`tcp-connect <target> <port>`	TCP 端口探测	适合测试 Web、数据库、业务端口
`source-interface <int>`	指定源接口	让探测报文使用该接口地址作为源地址
`source-ip <ip>`	指定源 IP	有些场景比源接口更明确
`frequency <seconds>`	探测间隔	每隔多少秒发起一次探测
`timeout <milliseconds>`	单次探测超时	超过这个时间没响应，本次探测失败
`threshold <milliseconds>`	响应时间阈值	常用于统计或告警，不等同于 Track 一定 Down
`ip sla schedule <id>`	启动任务	没有 schedule，任务不会真正运行
`track <id> ip sla <sla-id> reachability`	生成 Track 状态	把 IP SLA 结果变成 Up/Down

常见完整模板：

```
ip sla 10
 icmp-echo 8.8.8.8 source-interface Ethernet0/0
 frequency 5
 timeout 1000
 threshold 500
ip sla schedule 10 life forever start-time now

track 10 ip sla 10 reachability
 delay down 10 up 20
```

逐项理解：

配置	推荐理解
`frequency 5`	每 5 秒测一次
`timeout 1000`	每次最多等 1000ms
`threshold 500`	超过 500ms 算性能超过阈值，但不一定让 reachability Track Down
`delay down 10`	探测连续异常 10 秒后，Track 才 Down
`delay up 20`	探测恢复后连续正常 20 秒，Track 才 Up

`schedule` 也可以写得更具体：

```
ip sla schedule 10 life forever start-time now
ip sla schedule 10 life 3600 start-time now
```

写法	含义
`life forever`	一直运行
`life 3600`	运行 3600 秒后停止
`start-time now`	立即开始

排错时经常遇到一种情况：配置了 `ip sla 10` ，但忘了 `ip sla schedule 10 life forever start-time now` 。这时任务存在，但没有真正启动。

1.5 常见探测对象怎么选

探测对象选得不好，切换结果也会不好。

探测对象	优点	风险
本端直连下一跳	稳定、简单	只能证明到 ISP 边界可达， 不能证明互联网可达
公网 DNS	容易测试， 代表互联网连通性	可能被限速、过滤或策略影响
公司公网服务器	更贴近业务	服务器自身故障会导致出口误切
运营商指定探测地址	稳定性较好	只能代表运营商网络，不一定代表真实业务

课堂建议先用下一跳或公网 DNS 做实验，真实项目中再根据业务选择探测目标。

1.6 Track 防抖

链路质量差时，探测可能一会成功、一会失败。如果 Track 状态频繁 Up/Down，默认路由也会频繁切换。

可以给 Track 加延迟，避免抖动。

```
track 10 ip sla 10 reachability
 delay down 10 up 20
```

含义：

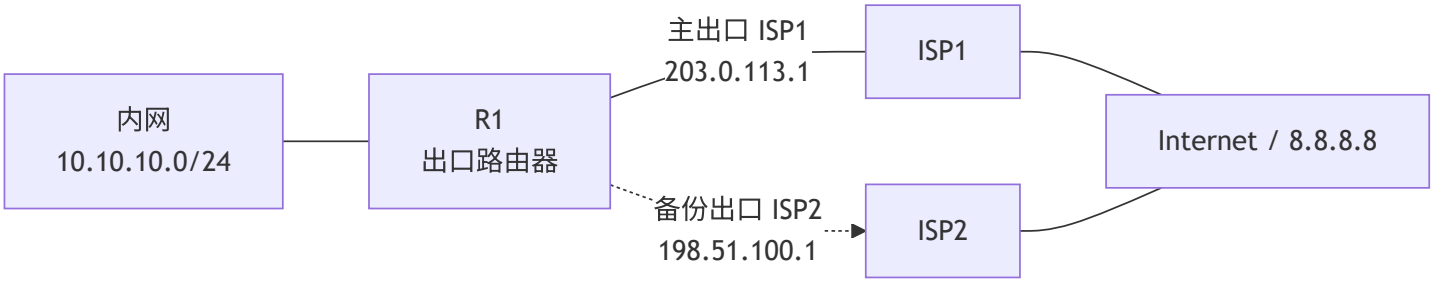
- 探测失败后，连续 10 秒仍然失败，Track 才 Down。
- 探测恢复后，连续 20 秒正常，Track 才 Up。

这能减少“刚坏就切、刚好就切回来”的震荡。

1.7 实战：双 ISP 默认路由切换

这个实验用 IP SLA 检测主出口是否可用，再用 Track 控制主默认路由。

拓扑：



需求：

- 正常走 ISP1。
- 探测 `8.8.8.8` 失败时撤销主默认路由。
- ISP2 的备份默认路由接管。

配置：

```
ip route 8.8.8.8 255.255.255.255 203.0.113.1

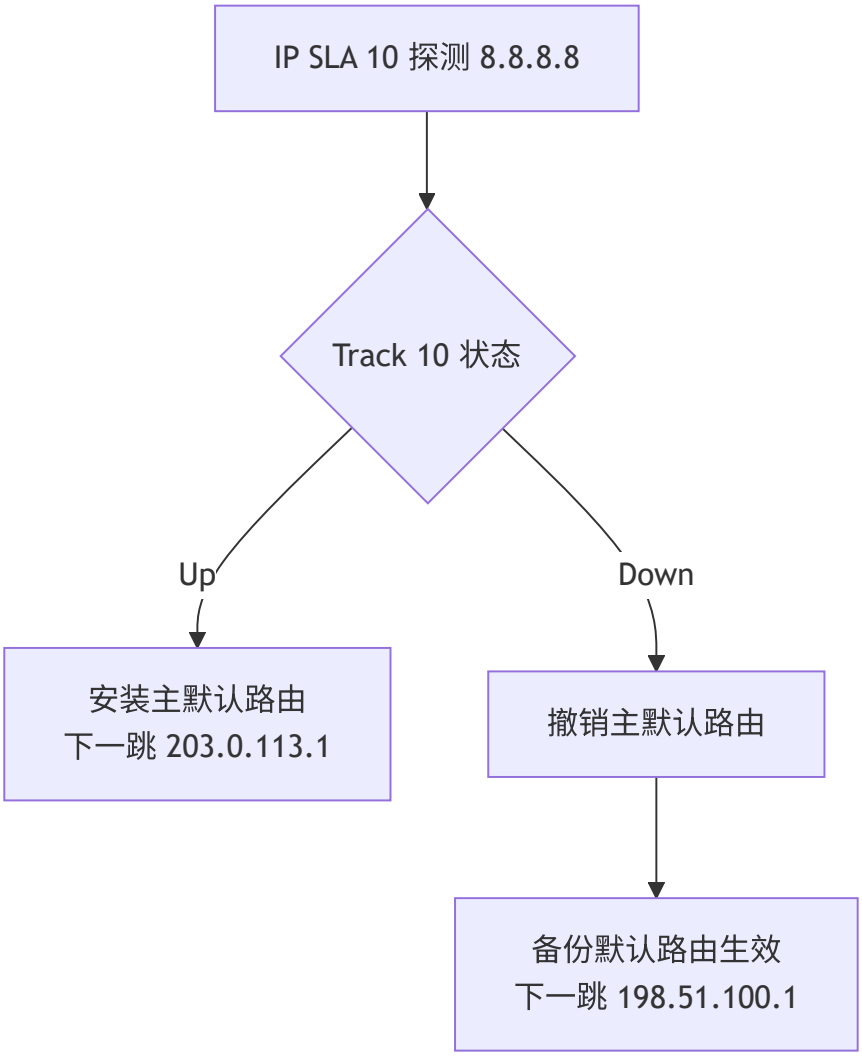
ip sla 10
 icmp-echo 8.8.8.8 source-interface Ethernet0/0
 frequency 5
 timeout 1000
ip sla schedule 10 life forever start-time now

track 10 ip sla 10 reachability
 delay down 10 up 20

ip route 0.0.0.0 0.0.0.0 203.0.113.1 track 10
ip route 0.0.0.0 0.0.0.0 198.51.100.1 200
```

逻辑：

配置	作用
`ip route 8.8.8.8/32 203.0.113.1`	让探测报文固定走 ISP1
`ip sla 10`	周期性探测 `8.8.8.8`
`track 10 ip sla 10 reachability`	把探测结果转换成 Track Up/Down
主默认路由 `track 10`	Track Up 时才安装
备份默认路由 AD 200	主默认路由撤销后才接管



验证：

```
show ip sla summary
show ip sla statistics
show track
show ip route 0.0.0.0
traceroute 8.8.8.8
```

设计注意：

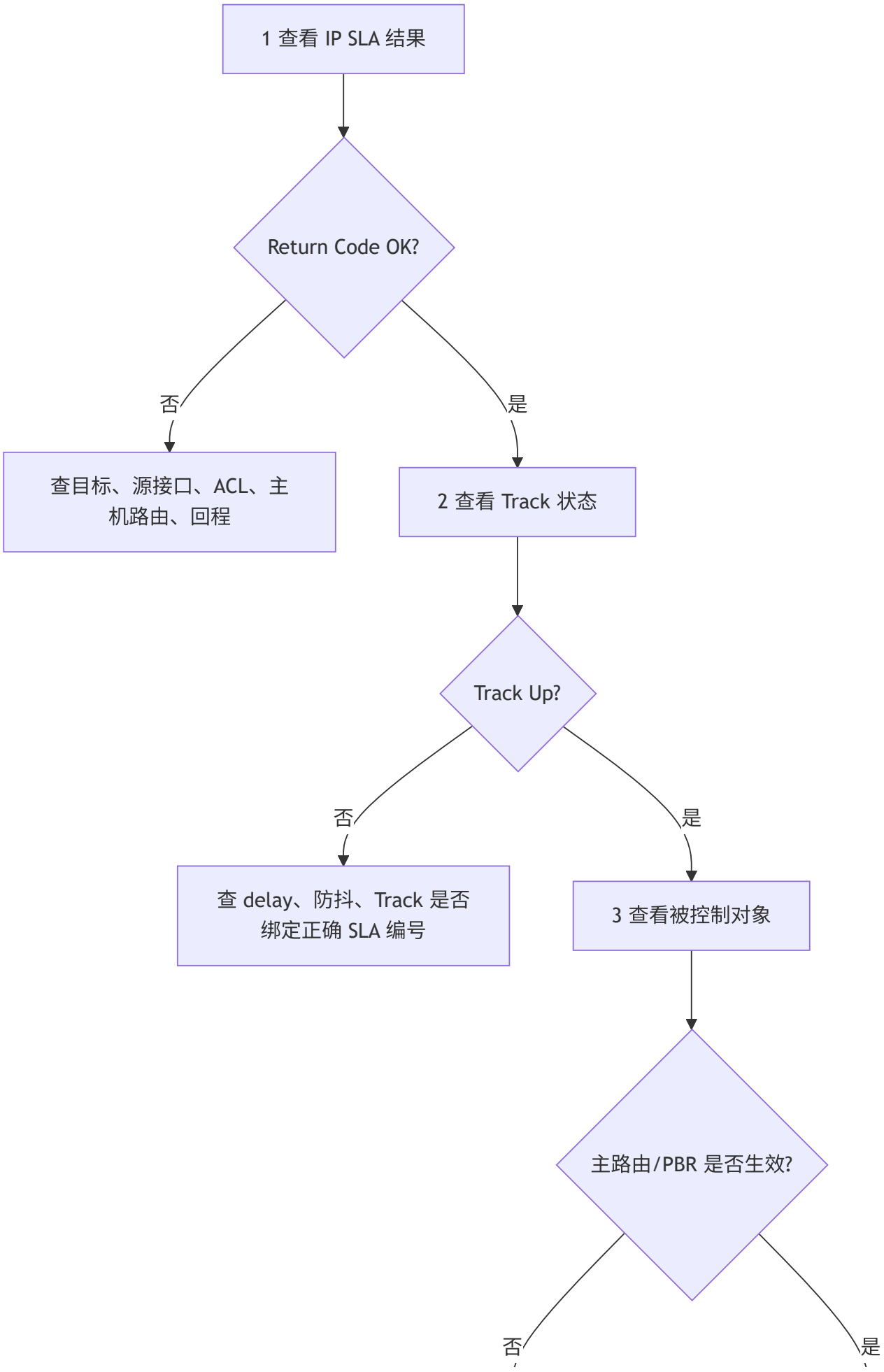
- 探测下一跳只能说明本地到 ISP 边界可达，不代表互联网或业务可达。
- 探测公网 DNS 或业务地址更接近真实效果，但要考虑远端限速和策略。
- 探测公网目标时，建议用 `/32` 主机路由固定探测路径，否则探测可能从备份出口绕出去。
- 频率越高切换越快，但控制流量和 CPU 开销越大。
- 源接口要明确，否则回程路径可能导致误判。

常见故障：

现象	可能原因	排查
`show track` 一直 Down	探测目标不可达、源接口不对、ACL 阻止 ICMP	`ping 8.8.8.8 source Ethernet0/0`
Track Up 但路由不生效	静态路由没有绑定 Track， 或有更优路由	`show ip route 0.0.0.0`
一直来回切换	探测目标不稳定，未配置 delay	`show ip sla statistics`

1.8 IP SLA 排错顺序

IP SLA 排错不要直接盯着路由表看，要顺着状态链往下查：


1 查看 IP SLA 结果
Return Code OK?
否
是
查目标、源接口、ACL、主机路由、回程
2 查看 Track 状态
Track Up?
否
是
查 delay、防抖、Track 是否绑定正确 SLA 编号
3 查看被控制对象
主路由/PBR 是否生效?
否
是



常用命令：

```
show ip sla configuration 10
show ip sla statistics 10
show track 10
show ip route 8.8.8.8
show ip route 0.0.0.0
ping 8.8.8.8 source Ethernet0/0
traceroute 8.8.8.8 source Ethernet0/0
```

2. BFD

BFD 是 Bidirectional Forwarding Detection，双向转发检测。

可以先把它理解成：两个邻居之间建立一个很轻量、很快的检测会话，用来快速发现对端或转发路径是否失效。

2.1 为什么需要 BFD

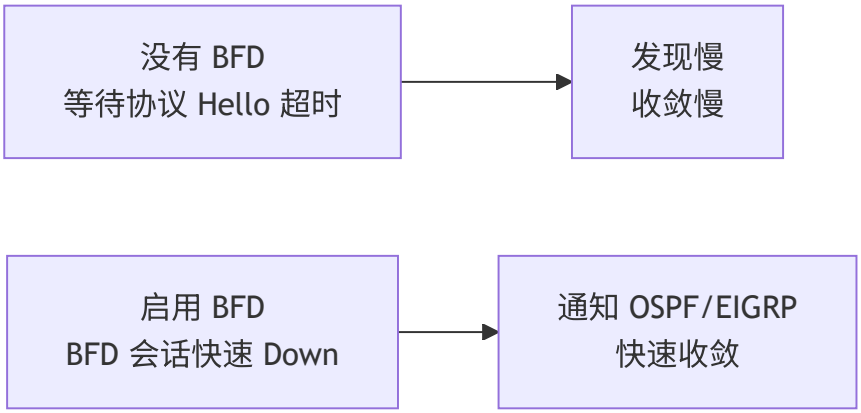
动态路由协议本身也有 Hello 和 Dead Timer，例如 OSPF 会定期发送 Hello。如果超过 Dead Timer 没收到 Hello，才认为邻居失效。

问题是：默认计时器往往比较慢。

协议	常见默认检测思路	可能的问题
OSPF	Hello / Dead Timer	默认可能要几十秒才发现故障
EIGRP	Hello / Hold Timer	收敛速度依赖 Hold Timer

如果链路物理 Down，接口会立刻 Down，协议也能马上知道。但如果是中间转发路径异常、单向故障、运营商二层故障，接口可能仍然 Up，协议就要等计时器超时。

BFD 的价值就是：用更快的检测机制告诉路由协议“邻居已经不可用了”，让路由协议尽快撤销路由。



2.2 BFD 和 IP SLA 的区别

对比项	BFD	IP SLA
本质	邻居之间的快速故障检测机制	路由器主动发起的探测任务
常见对象	路由协议邻居、转发路径	任意目标地址或业务目标
常见联动	OSPF、EIGRP、静态路由部分场景	Track、静态路由、PBR
典型速度	毫秒级到秒级	通常秒级
解决问题	快速发现邻居/路径故障	检测远端业务或目标是否可达
是否直接改路由	不直接改，通知上层协议	不直接改，要配合 Track

BFD 更适合动态路由协议快速收敛；IP SLA 更适合检测业务目标或远端连通性。

一句话区分：

- BFD 更像“邻居之间的快速心跳”。
- IP SLA 更像“路由器主动去测一个目标能不能访问”。

2.3 BFD 配置前先分清两层

BFD 至少要分成两层配置：

层次	配什么	作用
BFD 检测层	接口上的 `bfd interval ...`	定义 BFD 报文频率和检测时间
协议调用层	OSPF/EIGRP 调用 BFD	让路由协议在 BFD Down 时快速撤邻居或撤路由

只在接口上写 BFD 参数，不代表 OSPF 或 EIGRP 一定会使用 BFD。必须让上层协议调用它。



所以排错时要问两个问题：

1. BFD 会话有没有起来？
2. 路由协议有没有真的绑定 BFD？

如果第一个没起来，查接口、对端和平台支持。如果第一个起来了但协议不受影响，就查 OSPF/EIGRP 下有没有启用 BFD。

2.4 BFD 常用配置命令

BFD 的命令不要孤立背，要按这个顺序写：

1. 接口配置 BFD 检测参数
2. 路由协议调用 BFD
3. 用 show bfd neighbors 验证 BFD 会话
4. 模拟故障，看协议邻居是否快速 Down

接口 BFD 参数：

```
interface Ethernet0/0
 bfd interval 300 min_rx 300 multiplier 3
```

逐项解释：

参数	含义	例子
`interval 300`	本端期望发送 BFD 报文的最小间隔	本端大约每 300ms 发一次
`min_rx 300`	本端期望接收对端 BFD 报文的最小间隔	希望对端不要低于 300ms 发来
`multiplier 3`	连续丢失多少个 BFD 报文判定故障	连续 3 次没收到就 Down

大致检测时间：

```
检测时间 ≈ 协商后的接收间隔 × multiplier
```

例如两端都配置 `300/300/3`，可以先按大约 `900ms` 理解。实际结果要看两端协商，最终以 `show bfd neighbors details` 为准。

OSPF 调用 BFD 有两类常见方式。

方式一：在 OSPF 进程下全局启用，让该 OSPF 进程下支持 BFD 的接口都使用 BFD：

```
router ospf 1
 bfd all-interfaces
```

方式二：只在某个 OSPF 接口上启用：

```
interface Ethernet0/0
 ip ospf bfd
```

两种方式的区别：

方式	适合场景
`bfd all-interfaces`	实验或所有 OSPF 邻居都希望用 BFD
`ip ospf bfd`	只想让关键链路使用 BFD

EIGRP 调用 BFD 也分 classic 和 named mode。

Classic EIGRP：

```
router eigrp 100
 bfd all-interfaces
```

Named EIGRP：

```
router eigrp CCNP
 address-family ipv4 unicast autonomous-system 100
  af-interface Ethernet0/0
   bfd
  exit-af-interface
 exit-address-family
```

常用查看命令：

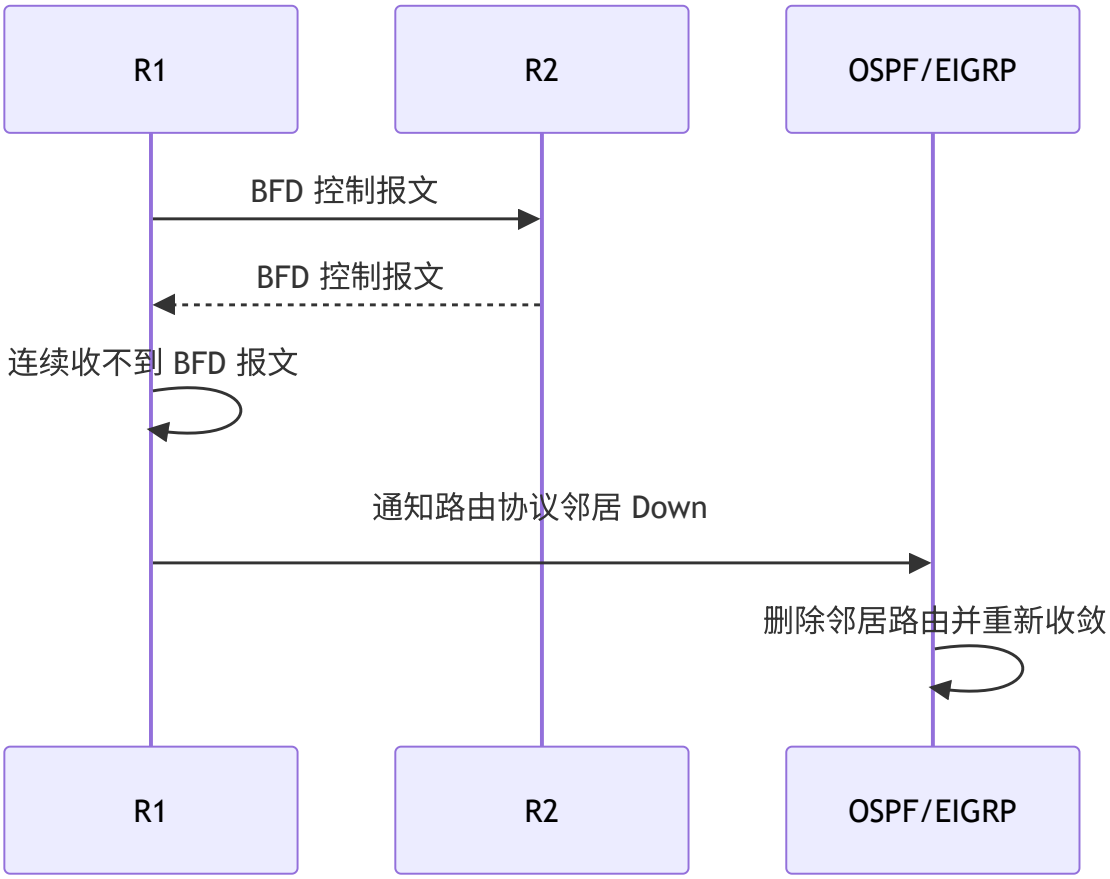
命令	看什么
`show bfd neighbors`	BFD 邻居是否 Up

命令	看什么
`show bfd neighbors details`	本地/远端参数、检测时间、会话状态
`show ip ospf neighbor`	OSPF 邻居是否受 BFD 影响
`show ip eigrp neighbors`	EIGRP 邻居是否受 BFD 影响
`show logging | include BFD`	故障时 BFD Down 的日志

注意：接口上只写 `bfd interval ...`，只是准备好了 BFD 检测参数；如果 OSPF/EIGRP 没有调用 BFD，协议邻居仍然会按自己的 Hello/Dead 或 Hold Timer 判断故障。

2.5 BFD 的基本工作方式

两台设备之间启用 BFD 后，会建立 BFD 会话，并周期性发送很小的 BFD 控制报文。



BFD 常见状态：

状态	含义
AdminDown	人为关闭
Down	会话未建立或检测失败

状态	含义
Init	已看到对端，但会话还没完全建立
Up	会话正常

常见计时参数：

参数	含义
`interval`	本端发送 BFD 报文的间隔
`min_rx`	本端希望接收对端 BFD 报文的最小间隔
`multiplier`	连续丢失多少个 BFD 报文后判定 Down

例如：

```
bfd interval 300 min_rx 300 multiplier 3
```

表示大约 300ms 发一次，连续 3 次没收到就认为故障，理论检测时间大约是：

```
300ms * 3 = 900ms
```

实际设备还会受平台性能、控制平面负载和邻居协商结果影响，不要在低端设备上盲目设置过小。

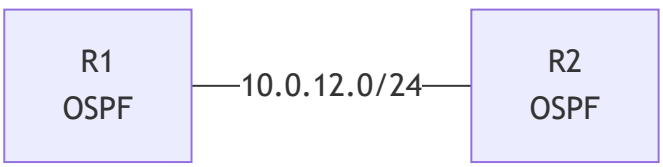
BFD 检测时间不是简单只看本地命令，还要看两端协商结果。初学可以先这样理解：

```
大致检测时间 = 协商后的接收间隔 × multiplier
```

例如两端都配置 `300/300/3` ，就按大约 900ms 理解。生产环境里不要只追求小数值，还要考虑 CPU、链路质量、设备平台和邻居数量。

2.6 BFD 与 OSPF 联动示例

拓扑：



配置思路：

1. 先让 OSPF 邻居正常建立。
2. 在接口上配置 BFD 检测参数。
3. 让 OSPF 使用 BFD 检测邻居状态。

示例：

R1：

```
interface Ethernet0/0
 ip address 10.0.12.1 255.255.255.0
 bfd interval 300 min_rx 300 multiplier 3
 ip ospf 1 area 0

router ospf 1
 router-id 1.1.1.1
 bfd all-interfaces
```

R2：

```
interface Ethernet0/0
 ip address 10.0.12.2 255.255.255.0
 bfd interval 300 min_rx 300 multiplier 3
 ip ospf 1 area 0

router ospf 1
 router-id 2.2.2.2
 bfd all-interfaces
```

有些 IOS 也可以不用 `bfd all-interfaces`，而是在具体 OSPF 接口上启用：

```
interface Ethernet0/0
 ip ospf bfd
```

两种方式不要混着背，核心是：接口上有 BFD 参数，OSPF 邻居又明确调用了 BFD。

效果：

- OSPF 邻居仍然由 OSPF 建立。
- BFD 不替代 OSPF，只负责更快发现邻居不可达。
- BFD Down 后，OSPF 会更快感知邻居 Down，从而重新计算路由。

验证：

```
R1# show ip ospf neighbor
R1# show bfd neighbors
R1# show bfd neighbors details
```

故障演示：

1. 正常时 OSPF Neighbor Full，BFD State Up。
2. 断开 R1-R2 中间链路，或在对端接口 shutdown。
3. 观察 BFD 先 Down。
4. OSPF 随后快速删除邻居。
5. 路由表中经 R2 学到的 OSPF 路由被撤销。

观察命令：

```
show logging | include BFD|OSPF
show ip route ospf
show ip ospf neighbor
show bfd neighbors details
```

2.7 BFD 与 EIGRP 联动

不同 IOS 版本命令略有差异，课堂实验前用 ？ 确认语法。

拓扑：



目标：

- R1/R2 正常建立 EIGRP 邻居。
- BFD 负责快速检测 R1-R2 转发路径。
- BFD Down 后，EIGRP 快速感知邻居不可用，撤销 `10.2.2.0/24` 。

Classic EIGRP 常见思路：

```
interface Ethernet0/0
 ip address 10.0.12.1 255.255.255.0
 bfd interval 300 min_rx 300 multiplier 3

router eigrp 100
 network 10.0.12.0 0.0.0.255
 bfd all-interfaces
```

Named EIGRP 的命令位置可能在 address-family 或 af-interface 下，例如：

```
router eigrp CCNP
 address-family ipv4 unicast autonomous-system 100
  network 10.0.12.0 0.0.0.255
  af-interface Ethernet0/0
   bfd
  exit-af-interface
 exit-address-family
```

配置重点不是背命令，而是理解联动关系：

```
BFD 发现故障 -> 通知 EIGRP/OSPF -> 协议撤销邻居和路由 -> 路径切换
```

验证：

```
show ip eigrp neighbors
show bfd neighbors
show ip route eigrp
show logging | include BFD|DUAL|EIGRP
```

如果 `show bfd neighbors` 是 Up，但 `show ip eigrp neighbors` 的收敛速度没有变化，通常说明 EIGRP 没有真正调用 BFD。

2.8 BFD 验证与排错

常用查看命令：

```
show bfd neighbors
show bfd neighbors details
show ip ospf neighbor
show ip eigrp neighbors
```

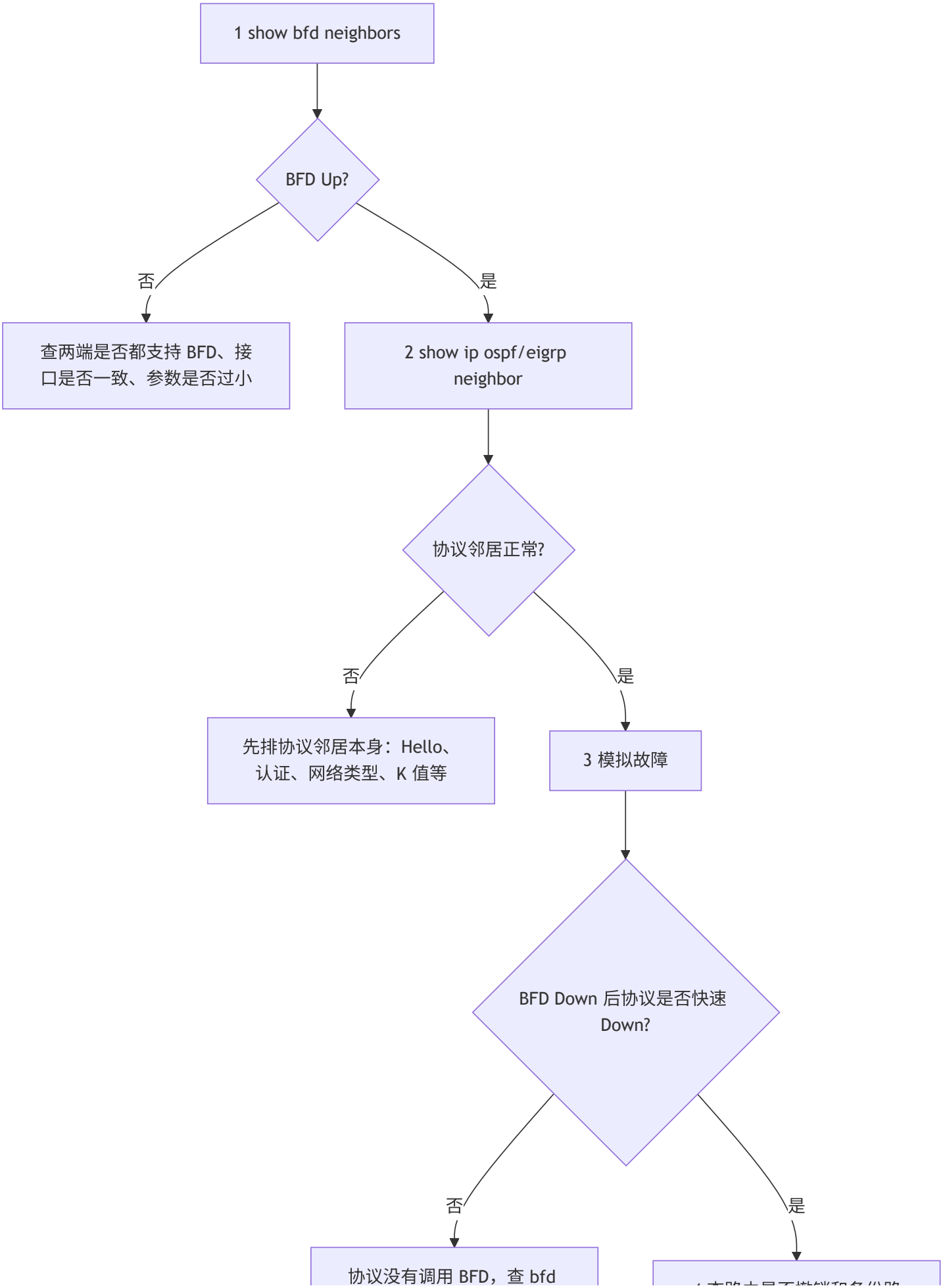
验证重点：

- BFD 邻居是否 Up。
- BFD 绑定的是哪个接口、哪个邻居。
- 路由协议邻居是否也正常。
- 故障时 BFD 是否先 Down，然后路由协议邻居 Down。

常见问题：

现象	可能原因	排查
BFD 不建立	两端没都启用、接口不匹配、 平台不支持	查接口配置和 `show bfd neighbors`
BFD Up 但协议不受影响	路由协议没有调用 BFD	查 OSPF/EIGRP 下的 BFD 配置
BFD 频繁 Down	计时器太小、设备负载高、 链路质量差	放大 interval/min_rx/multiplier
命令不存在	IOS 版本或镜像不支持	用 `?` 查看平台支持

BFD 排错流程：


1 show bfd neighbors
BFD Up?
否
是
查两端是否都支持 BFD、接口是否一致、参数是否过小
2 show ip ospf/eigrp neighbor
协议邻居正常?
否
是
先排协议邻居本身：Hello、认证、网络类型、K 值等
3 模拟故障
BFD Down 后协议是否快速 Down?
否
是
协议没有调用 BFD，查 bfd

all-interfaces / ip ospf bfd / af-interface bfd	4 查路由是否撤销和备份路径是否接管

看日志时要注意顺序：

```
BFD Down -> OSPF/EIGRP Neighbor Down -> 路由撤销 -> 备用路径安装
```

如果先看到协议邻居超时，隔很久才看到 BFD，说明 BFD 可能没有真正参与这个邻居的检测。

设计建议：

- 初学实验不要直接上 50ms，可以先用 300ms 或 500ms。
- 两端计时器尽量一致，便于理解和排错。
- 只在关键邻居上启用 BFD，不要全网无脑启用极小计时器。
- BFD 解决快速发现邻居故障，不负责判断远端业务是否可用；远端业务检测仍然更适合 IP SLA。

3. PBR

PBR 是 Policy-Based Routing，策略路由。它允许路由器在普通查路由表之前，根据策略决定流量下一跳。

PBR 常见匹配条件：

- 源地址。
- 目的地址。
- 协议类型。
- TCP/UDP 端口。
- DSCP/ToS。
- 输入接口。

PBR 常见动作：

- `set ip next-hop`
- `set ip default next-hop`
- `set interface`
- `set ip precedence`
- `set ip next-hop verify-availability`

PBR 的核心结构：

```
匹配流量 -> route-map 设置动作 -> 入接口调用 ip policy route-map
```

PBR 必须应用在流量进入路由器的接口上。

3.1 PBR permit 与 deny

PBR 中 route-map 的 `permit/deny` 行为容易误解。

Route-map 结果	PBR 行为
匹配 `permit` 且 set 可用	按策略转发
匹配 `deny`	不走策略，回到普通路由表
没匹配任何条目	回到普通路由表

示例：

```
route-map PBR deny 10
 match ip address 101

route-map PBR permit 20
 match ip address 102
 set ip next-hop 192.0.2.2
```

匹配 ACL 101 的流量不会被丢弃，而是普通查表。PBR 不是 ACL 丢包工具。

如果确实要让未匹配流量被丢弃，可以显式把它送到 Null0，但要非常谨慎。

```
route-map PBR permit 100
 set interface Null0
```

3.2 PBR 常见 set 动作

3.2.1 set ip next-hop

命中策略后直接指定下一跳。

```
route-map PBR permit 10
 match ip address 101
 set ip next-hop 203.0.113.1
```

可配置多个下一跳，按顺序尝试：

```
route-map PBR permit 10
 match ip address 101
 set ip next-hop 203.0.113.1 198.51.100.1
```

3.2.2 set ip default next-hop

只有普通路由表没有明确匹配路由时才使用策略下一跳。

```
route-map PBR permit 10
 match ip address 101
 set ip default next-hop 203.0.113.1
```

命令	行为
`set ip next-hop`	命中 PBR 后优先按策略走
`set ip default next-hop`	普通路由没有可用路径时才使用策略

3.2.3 set interface

指定出接口。

```
route-map PBR permit 10
 match ip address 101
 set interface Serial0/0
```

点到点接口可用性较高，多访问网络不建议只写出接口，因为二层下一跳不明确。

4. Local PBR

接口 PBR 只影响从接口进入路由器的流量，不影响路由器本机产生的流量。

本机流量包括：

- 路由器自己 ping。
- SNMP。
- Syslog。
- NTP。
- AAA。
- 设备自己发起的管理连接。

控制本机流量使用 Local PBR：

```
ip access-list extended LOCAL-MGMT
 permit ip host 10.255.255.1 any

route-map LOCAL-PBR permit 10
 match ip address LOCAL-MGMT
 set ip next-hop 203.0.113.1

ip local policy route-map LOCAL-PBR
```

Local PBR 配错可能影响设备管理流量，生产环境要谨慎。

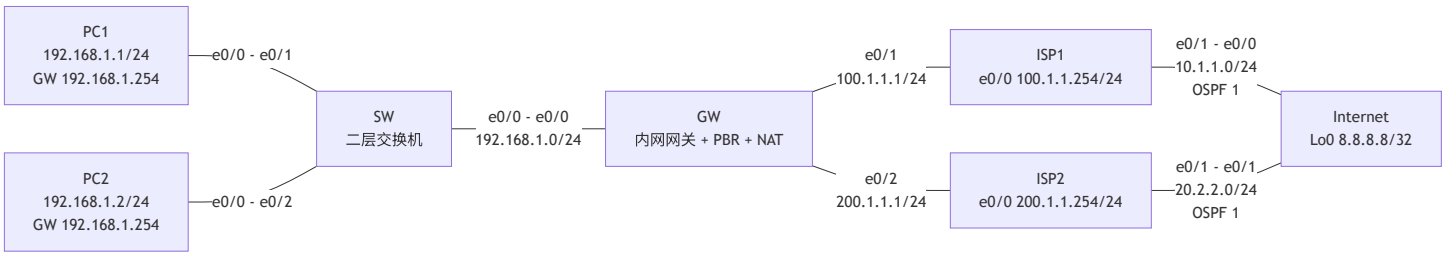
5. 综合实战：OSPF + BFD + IP SLA + PBR + NAT

这个实验把本章几个路径控制工具串起来，但拓扑上分清两件事：

- 内网侧：PC1、PC2 接入二层交换机，GW 作为统一网关，使用 PBR 按源地址分流。
- 运营商侧：ISP1、ISP2 与 Internet 之间运行 OSPF，Internet 上用 `8.8.8.8/32` 模拟公网目标。

BFD 加在 ISP1-Internet、ISP2-Internet 的 OSPF 邻居链路上，用来观察链路故障时 OSPF 邻居更快 Down。GW 不运行 OSPF，所以 GW 到 ISP1/ISP2 的出口切换仍然靠 IP SLA + Track。

5.1 拓扑



地址规划：

设备/接口	地址	说明
PC1-E0/0	`192.168.1.1/24`	网关 `192.168.1.254`
PC2-E0/0	`192.168.1.2/24`	网关 `192.168.1.254`
GW-E0/0	`192.168.1.254/24`	内网网关，PBR 入接口，NAT inside
GW-E0/1	`100.1.1.1/24`	连接 ISP1，NAT outside

设备/接口	地址	说明
GW-E0/2	`200.1.1.1/24`	连接 ISP2，NAT outside
ISP1-E0/0	`100.1.1.254/24`	连接 GW，OSPF 被动宣告
ISP1-E0/1	`10.1.1.1/24`	连接 Internet，建立 OSPF 邻居
ISP2-E0/0	`200.1.1.254/24`	连接 GW，OSPF 被动宣告
ISP2-E0/1	`20.2.2.1/24`	连接 Internet，建立 OSPF 邻居
Internet-E0/0	`10.1.1.254/24`	连接 ISP1
Internet-E0/1	`20.2.2.254/24`	连接 ISP2
Internet-Loopback0	`8.8.8.8/32`	公网测试目标

5.2 实验目标

正常状态：

- PC1 源地址 `192.168.1.1` 命中 PBR，优先走 ISP1。
- PC2 源地址 `192.168.1.2` 命中 PBR，优先走 ISP2。
- GW 的 E0/1、E0/2 都配置 NAT outside，内网流量从哪个出口出去，就转换成对应出口接口地址。
- ISP1、ISP2、Internet 之间运行 OSPF，Internet 能通过 OSPF 学到 `100.1.1.0/24` 和 `200.1.1.0/24`，保证 NAT 回包可达。
- ISP1-Internet、ISP2-Internet 的 OSPF 邻居调用 BFD，加快邻居故障检测。

故障状态：

- ISP1 探测失败时，Track 10 Down，GW 的主默认路由撤销，普通流量可以切到 ISP2。
- ISP2 探测失败时，Track 20 Down，PC2 的 PBR 下一跳不可用，PC2 回退到普通路由表。
- ISP 到 Internet 的 OSPF 链路故障时，BFD 先 Down，随后 OSPF 邻居快速 Down。

5.3 配置顺序总览

顺序	配置内容	目的
第一步	配 PC、SW、GW、ISP、Internet 基础地址	先保证直连接口和网关正常
第二步	配 ISP1、ISP2、Internet 的 OSPF 与 BFD	让 Internet 侧知道两个出口网段，并让 OSPF 邻居快速感知故障

顺序	配置内容	目的
第三步	配 GW 的 IP SLA、Track 和默认路由	让普通默认路由具备主备切换能力
第四步	配 GW 双出口 NAT	保证内网访问公网能回包
第五步	配 GW PBR	按 PC 源地址选择 ISP1 或 ISP2
第六步	验证正常路径和故障切换	分层确认 OSPF、BFD、NAT、PBR、Track 生效

配置顺序不要反过来。先让底层地址和 ISP 侧 OSPF 正常，再做 GW 上的路径控制。PBR 最后再挂到入接口，方便排错。

5.4 第一步：配置基础接口

SW 只做二层转发。PC1、PC2 和 GW 都在同一个二层网段，所以这里不需要 VLAN 单臂路由。

SW：

```
interface Ethernet0/0
 description TO-GW
 switchport mode access
 spanning-tree portfast
 no shutdown

interface Ethernet0/1
 description TO-PC1
 switchport mode access
 spanning-tree portfast
 no shutdown

interface Ethernet0/2
 description TO-PC2
 switchport mode access
 spanning-tree portfast
 no shutdown
```

GW：

```
interface Ethernet0/0
 description TO-SW-LAN
 ip address 192.168.1.254 255.255.255.0
 ip nat inside
 no shutdown

interface Ethernet0/1
 description TO-ISP1
 ip address 100.1.1.1 255.255.255.0
 ip nat outside
 no shutdown

interface Ethernet0/2
 description TO-ISP2
 ip address 200.1.1.1 255.255.255.0
 ip nat outside
 no shutdown
```

PC1：

```
IP address: 192.168.1.1
Mask:       255.255.255.0
Gateway:    192.168.1.254
```

PC2：

```
IP address: 192.168.1.2
Mask:       255.255.255.0
Gateway:    192.168.1.254
```

ISP1：

```
interface Ethernet0/0
 description TO-GW
 ip address 100.1.1.254 255.255.255.0
 no shutdown

interface Ethernet0/1
 description TO-INTERNET
 ip address 10.1.1.1 255.255.255.0
 ip ospf network point-to-point
 no shutdown
```

ISP2：

```
interface Ethernet0/0
 description TO-GW
 ip address 200.1.1.254 255.255.255.0
 no shutdown

interface Ethernet0/1
 description TO-INTERNET
 ip address 20.2.2.1 255.255.255.0
 ip ospf network point-to-point
 no shutdown
```

Internet：

```
interface Ethernet0/0
 description TO-ISP1
 ip address 10.1.1.254 255.255.255.0
 ip ospf network point-to-point
 no shutdown

interface Ethernet0/1
 description TO-ISP2
 ip address 20.2.2.254 255.255.255.0
 ip ospf network point-to-point
 no shutdown

interface Loopback0
 description INTERNET-TEST
 ip address 8.8.8.8 255.255.255.255
 ip ospf network point-to-point
```

基础连通验证：

```
PC1> ping 192.168.1.254
PC2> ping 192.168.1.254
GW# ping 100.1.1.254
GW# ping 200.1.1.254
ISP1# ping 10.1.1.254
ISP2# ping 20.2.2.254
```

5.5 第二步：配置 ISP 侧 OSPF 与 BFD

这里的 OSPF 只运行在 ISP1、ISP2、Internet 三台设备之间。GW 不跑 OSPF，GW 通过静态默认路由把内网流量送给 ISP。

BFD 只加在真正建立 OSPF 邻居的链路上：

BFD 链路	设备接口
ISP1-Internet	ISP1 E0/1 与 Internet E0/0
ISP2-Internet	ISP2 E0/1 与 Internet E0/1

这里不用 `bfd all-interfaces`，而是在具体接口上使用 `ip ospf bfd`，这样更容易看清楚 BFD 到底绑定在哪条邻居链路。

ISP1：

```
router ospf 1
 router-id 1.1.1.1
 passive-interface default
 no passive-interface Ethernet0/1

interface Ethernet0/0
 ip ospf 1 area 0

interface Ethernet0/1
 bfd interval 300 min_rx 300 multiplier 3
 ip ospf 1 area 0
 ip ospf bfd
```

ISP2：

```
router ospf 1
 router-id 2.2.2.2
 passive-interface default
 no passive-interface Ethernet0/1

interface Ethernet0/0
 ip ospf 1 area 0

interface Ethernet0/1
 bfd interval 300 min_rx 300 multiplier 3
 ip ospf 1 area 0
 ip ospf bfd
```

Internet：

```
router ospf 1
 router-id 8.8.8.8
 passive-interface default
 no passive-interface Ethernet0/0
 no passive-interface Ethernet0/1

interface Ethernet0/0
 bfd interval 300 min_rx 300 multiplier 3
 ip ospf 1 area 0
 ip ospf bfd

interface Ethernet0/1
 bfd interval 300 min_rx 300 multiplier 3
 ip ospf 1 area 0
 ip ospf bfd

interface Loopback0
 ip ospf 1 area 0
```

验证 OSPF 与 BFD：

```
ISP1# show ip ospf neighbor
ISP1# show bfd neighbors
ISP1# show bfd neighbors details
ISP2# show ip ospf neighbor
ISP2# show bfd neighbors
Internet# show ip ospf neighbor
Internet# show bfd neighbors
Internet# show ip route ospf
```

预期：

- ISP1 与 Internet 建立 OSPF 邻居。
- ISP2 与 Internet 建立 OSPF 邻居。
- ISP1-Internet、ISP2-Internet 的 BFD 邻居状态为 Up。
- Internet 能学到 `100.1.1.0/24` 和 `200.1.1.0/24`，这样 NAT 后的回包可以回到对应 ISP。

5.6 第三步：GW 配置 IP SLA、Track 与默认路由

GW 使用 IP SLA 生成两个 Track：

Track	探测对象	作用
Track 10	`8.8.8.8`，源接口 E0/1	控制 ISP1 主默认路由
Track 20	ISP2 下一跳 `200.1.1.254`，源接口 E0/2	控制 PC2 的 PBR 下一跳是否可用

GW：

```
ip route 8.8.8.8 255.255.255.255 100.1.1.254

ip sla 10
 icmp-echo 8.8.8.8 source-interface Ethernet0/1
 frequency 5
 timeout 1000
ip sla schedule 10 life forever start-time now

track 10 ip sla 10 reachability
 delay down 10 up 20

ip sla 20
 icmp-echo 200.1.1.254 source-interface Ethernet0/2
 frequency 5
 timeout 1000
ip sla schedule 20 life forever start-time now

track 20 ip sla 20 reachability
 delay down 10 up 20

ip route 0.0.0.0 0.0.0.0 100.1.1.254 track 10
ip route 0.0.0.0 0.0.0.0 200.1.1.254 200
```

这里有一个关键点： `8.8.8.8/32` 的主机路由固定指向 ISP1，目的是让 ISP1 的探测始终从 ISP1 方向出去。否则主默认路由撤销后，探测报文可能绕到 ISP2，又把 Track 10 拉 Up，造成反复震荡。

Track 20 这里探测的是 ISP2 下一跳，不是 Internet 后端业务。它适合判断 GW 到 ISP2 的出口是否可用；如果要判断 ISP2 后面的公网可达性，需要额外设计一个只走 ISP2 的探测目标或探测路径。

验证：

```
GW# show ip sla statistics 10
GW# show ip sla statistics 20
GW# show track
GW# show ip route 0.0.0.0
GW# show ip route 8.8.8.8
```

5.7 第四步：GW 配置双出口 NAT

GW 有两个公网方向出口，所以两个出口都配置 NAT outside。推荐使用 route-map NAT，让流量从哪个接口出去，就转换成哪个接口地址。

```
ip access-list standard NAT-LAN
 permit 192.168.1.0 0.0.0.255

route-map NAT-ISP1 permit 10
 match ip address NAT-LAN
 match interface Ethernet0/1

route-map NAT-ISP2 permit 10
 match ip address NAT-LAN
 match interface Ethernet0/2

ip nat inside source route-map NAT-ISP1 interface Ethernet0/1 overload
ip nat inside source route-map NAT-ISP2 interface Ethernet0/2 overload
```

如果平台不支持 NAT route-map 里的 `match interface`，可以退一步按源地址分别 NAT：PC1 NAT 到 E0/1，PC2 NAT 到 E0/2。但这种写法在故障回退时不如按出接口 NAT 灵活。

5.8 第五步：GW 配置 PBR

PBR 应用在 GW 的内网入接口 E0/0，因为 PC1、PC2 的流量都是从这里进入 GW。

需求：

- 源地址 `192.168.1.1` 的流量走 ISP1，下一跳 `100.1.1.254`。
- 源地址 `192.168.1.2` 的流量走 ISP2，下一跳 `200.1.1.254`。
- 如果 PC1/PC2 对应的 Track Down，就不强制 PBR，回退查普通路由表。

```
ip access-list extended PC1-TRAFFIC
 permit ip host 192.168.1.1 any

ip access-list extended PC2-TRAFFIC
 permit ip host 192.168.1.2 any

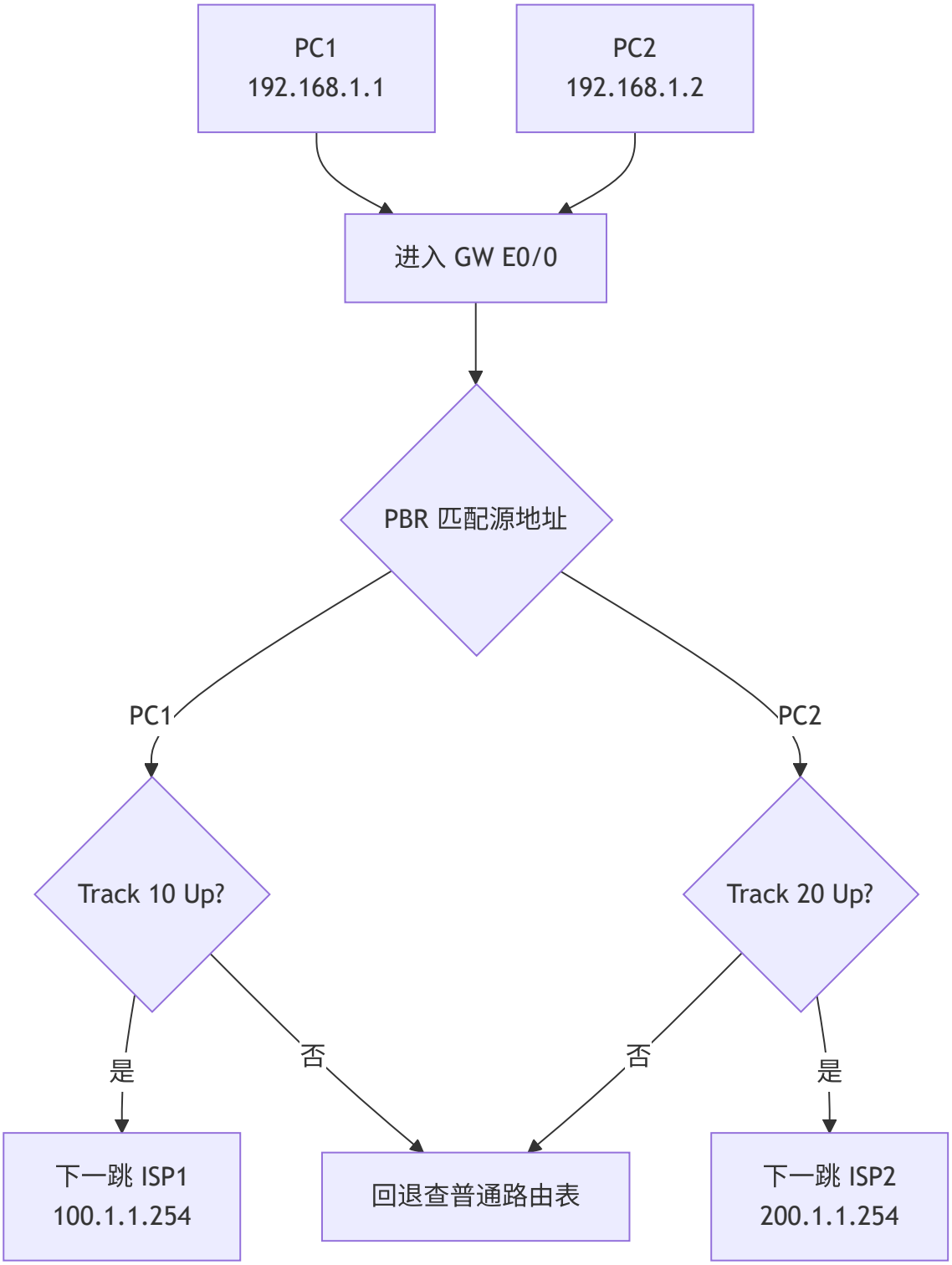
route-map LAN-PBR permit 10
 match ip address PC1-TRAFFIC
 set ip next-hop verify-availability 100.1.1.254 1 track 10

route-map LAN-PBR permit 20
 match ip address PC2-TRAFFIC
 set ip next-hop verify-availability 200.1.1.254 1 track 20

route-map LAN-PBR permit 100

interface Ethernet0/0
 ip policy route-map LAN-PBR
```

逻辑：



注意： `set ip next-hop verify-availability` 的具体支持情况与 IOS 镜像有关。如果实验镜像不支持，可以临时改成：

```
route-map LAN-PBR permit 10
 match ip address PC1-TRAFFIC
 set ip next-hop 100.1.1.254

route-map LAN-PBR permit 20
 match ip address PC2-TRAFFIC
 set ip next-hop 200.1.1.254
```

但普通 `set ip next-hop` 不会根据 Track 自动失效，链路坏了时仍可能把流量硬送给不可用下一跳。

5.9 第六步：正常验证

先验证 ISP 侧 OSPF 与 BFD：

```
ISP1# show ip ospf neighbor
ISP1# show bfd neighbors
ISP1# show bfd neighbors details

ISP2# show ip ospf neighbor
ISP2# show bfd neighbors
ISP2# show bfd neighbors details

Internet# show ip ospf neighbor
Internet# show bfd neighbors
```

再验证 GW 的路径控制状态：

```
GW# show ip policy
GW# show route-map LAN-PBR
GW# show access-lists PC1-TRAFFIC
GW# show access-lists PC2-TRAFFIC
GW# show track
GW# show ip route 0.0.0.0
GW# show ip nat translations
```

业务验证：

```
PC1> ping 8.8.8.8
PC2> ping 8.8.8.8
PC1> trace 8.8.8.8
PC2> trace 8.8.8.8
GW# show ip nat translations
```

预期：

- PC1 访问 `8.8.8.8` 时，下一跳走 ISP1 `100.1.1.254`。
- PC2 访问 `8.8.8.8` 时，下一跳走 ISP2 `200.1.1.254`。
- GW 上能看到 PC1 被 NAT 成 `100.1.1.1`，PC2 被 NAT 成 `200.1.1.1`。
- Internet 的回包能通过 OSPF 回到 ISP1 或 ISP2，再回到 GW。

如果 PC 使用 Windows 或 Linux，路径测试命令对应为：

```
tracert 8.8.8.8
traceroute 8.8.8.8
```

5.10 故障演示

故障一：模拟 ISP1 出口不可用。

在 GW 上关闭 ISP1 出口：

```
interface Ethernet0/1
 shutdown
```

观察：

```
show ip sla statistics 10
show track 10
show ip route 0.0.0.0
show route-map LAN-PBR
```

预期：

- Track 10 Down。
- 指向 ISP1 的主默认路由撤销。
- 备份默认路由指向 ISP2。
- PC1 的 PBR 下一跳不可用，回退普通路由表后走 ISP2。

恢复 ISP1：

```
interface Ethernet0/1
 no shutdown
```

故障二：模拟 ISP2 出口不可用。

在 GW 上关闭 ISP2 出口：

```
interface Ethernet0/2
 shutdown
```

观察：

```
show ip sla statistics 20
show track 20
show route-map LAN-PBR
show ip nat translations
```

预期：

- Track 20 Down。
- PC2 的 PBR 下一跳不可用。
- PC2 回退普通路由表，正常情况下会走 ISP1 主默认路由。

恢复 ISP2：

```
interface Ethernet0/2
 no shutdown
```

故障三：模拟 ISP 到 Internet 的 OSPF 链路故障。

比如关闭 ISP2 到 Internet 的接口：

```
ISP2(config)# interface Ethernet0/1
ISP2(config-if)# shutdown
```

观察：

```
ISP2# show ip ospf neighbor
ISP2# show bfd neighbors
ISP2# show bfd neighbors details
Internet# show ip route ospf
Internet# show bfd neighbors
GW# show track 20
PC2> ping 8.8.8.8
```

BFD 启用后，ISP2 与 Internet 之间的 BFD 会话会先 Down，随后 OSPF 邻居快速 Down。这个现象比单纯等待 OSPF Dead Timer 更明显。

如果实验里直接 `shutdown` 物理接口，接口自身 Down 也会让 OSPF 立即撤邻居，所以不一定能完全体现 BFD 的价值。更适合观察 BFD 的方式是让二层接口保持 Up，但让邻居报文或转发路径异常；课堂上可以先看 `show bfd neighbors details` 确认 BFD 会话存在，再对比关闭 BFD 前后的邻居故障感知时间。

还要注意：GW 的 Track 20 只探测 `200.1.1.254` 。如果只是 ISP2 到 Internet 的 E0/1 链路断了，而 GW 到 ISP2 的 E0/2 仍然可达，Track 20 仍可能保持 Up。这个现象正好可以用来说明：IP SLA 探测对象必须和你真正关心的故障点一致。

5.11 排错顺序

这个综合实验排错时按五层看：

1. PC 到 GW 网关是否通
2. GW 到 ISP1/ISP2 下一跳是否通
3. ISP1/ISP2 到 Internet 的 OSPF 是否正常
4. BFD 邻居是否 Up，OSPF 是否真正调用 BFD
5. GW 的 IP SLA / Track 是否符合预期
6. GW 的 PBR 和 NAT 是否命中

对应命令：

```
show ip interface brief
show ip ospf neighbor
show bfd neighbors
show bfd neighbors details
show logging | include BFD|OSPF
show ip route ospf
show ip sla statistics
show track
show ip route 0.0.0.0
show ip policy
show route-map
show access-lists
show ip nat translations
show ip nat statistics
```